Quiz Master – Problem Statement & Wireframes

Oct 19, 2026 · @Aaron

Overview

Build **Quiz Master**, a multi-user exam-preparation web app. An administrator (the Quiz Master) creates subjects, chapters and timed multiple-choice quizzes; registered users attempt the quizzes, get scored instantly and track their progress.

Roles

Role	How they get access	Main purpose
Admin (Quiz Master)	Created automatically when the database is set up. There is only one admin and no admin sign-up.	Manage all content and users, view summaries, run reports
User	Registers with email and password, then logs in	Attempt quizzes, view scores and progress, export own data

Terms used

Term	Meaning
Subject	A course or topic area, e.g. Python
Chapter	A part of a subject, e.g. Data Structures. A subject has many chapters.
Quiz	A timed test inside a chapter, with a date and a duration. A chapter has many quizzes.
Question	A multiple-choice question with 2–4 options and exactly one correct option. A quiz has many questions.
Score	One attempt of one quiz by one user: marks scored, total questions, time of attempt

Frameworks to be used

Layer	Must use
Backend API	Flask (JSON APIs only – no HTML pages rendered by Flask for the UI)
Database	SQLite, created by code (no manual database creation)
Frontend	Vue 3 with the Options API, Vue Router, Vuex; Vite for the project setup
Styling	Bootstrap 5 (installed with npm)
Login	Token-based authentication (Flask-Security-Too) with role-based access
Background jobs	Celery with Redis as the message broker
Caching	Redis (Flask-Caching)
Templates	Jinja2 only for the HTML of emails/reports, not for the UI

Core functionalities

1. Login and role-based access

- Users register with email (username), password, full name, qualification and date of birth; admin does not register.
- One login form for both roles. After login the app shows the admin or user dashboard based on the role.
- Every API checks the token; admin APIs also check the admin role. A user must not be able to open admin pages or call admin APIs.
- Logout ends the session. Blocked users cannot log in.

2. Admin dashboard

- Create, edit and delete **subjects**.
- Create, edit and delete **chapters** inside a subject.
- Create, edit and delete **quizzes** inside a chapter, each with a date and a time duration.

- Create, edit and delete **questions** inside a quiz: question text, up to 4 options, and the one correct option.
- View all users and block or unblock a user.
- **Search** users, subjects and quizzes by name.
- **Summary charts**, e.g. top score and number of attempts per subject.

3. User dashboard

- See the available quizzes (subject, chapter, date, number of questions, duration), filter by subject.
- **Attempt a quiz** with a visible countdown timer; the quiz is submitted automatically when time runs out.
- The score is calculated on the server and saved; the correct answers are never sent to the browser.
- View past scores and **summary charts** of their own performance.

4. Backend jobs (Celery + Redis)

Job	Type	What it does
Daily reminder	Scheduled, every evening	Email users who have not visited in a day, or tell them about new quizzes
Monthly activity report	Scheduled, 1st of every month	HTML email per user: quizzes attempted, average score, rank, list of attempts
Export as CSV	Triggered by a button, runs in the background	User: their quiz attempts. Admin: per-user statistics. The user is notified when the file is ready to download.

5. Performance and caching

- Cache frequently read data (subject list, quiz list, summaries) in Redis.
- Every cached entry must expire, and must be cleared when the data changes.
- APIs should stay fast as data grows.

Optional functionalities

- Charts drawn with a JavaScript library such as ApexCharts (vue3-apexcharts).
- One responsive UI that works on both mobile and desktop.
- Form validation on the frontend (HTML5 attributes or JavaScript) **and** on the backend.
- Clean, consistent styling with Bootstrap.
- A PDF version of the monthly activity report, as an alternative to HTML.
- Admin buttons to run the scheduled jobs immediately (useful for demos).

Data model

The minimum tables; add more fields if your app needs them.

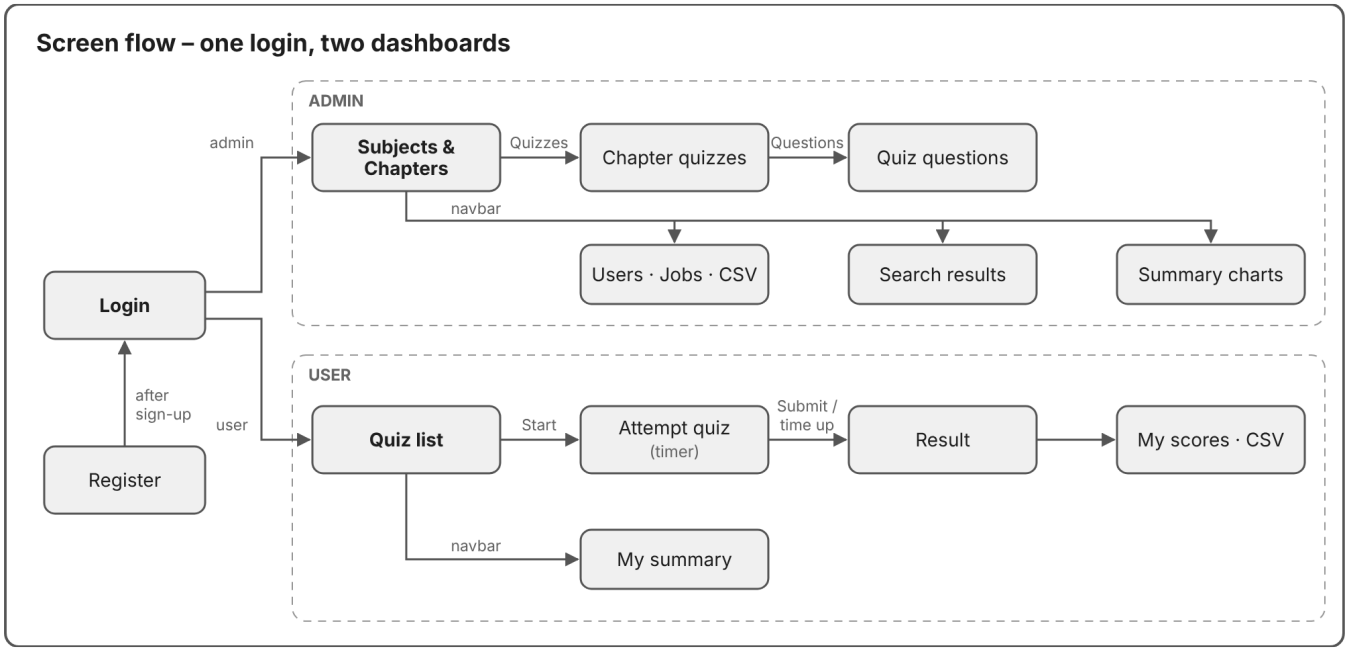
Table	Main fields
User	id, email, password (hashed), full_name, qualification, dob, active, roles
Role	id, name (admin / user)
Subject	id, name, description
Chapter	id, subject_id, name, description
Quiz	id, chapter_id, title, date_of_quiz, time_duration, remarks
Question	id, quiz_id, statement, option1–option4, correct_option
Score	id, quiz_id, user_id, time_stamp_of_attempt, total_scored, total_questions

Deleting a subject deletes its chapters, quizzes, questions and scores.

Frontend wireframes (UI)

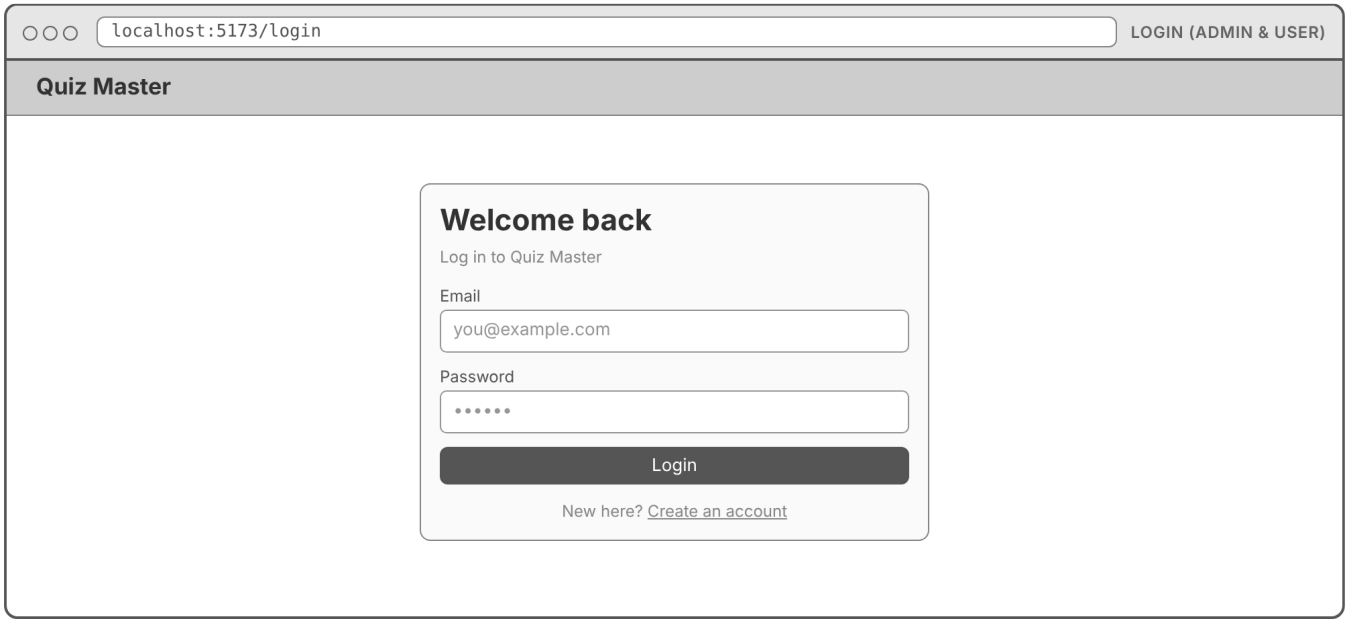
Low-fidelity layouts of every screen. They show what each page contains and where; colours and exact styling are up to you (Bootstrap).

Screen flow

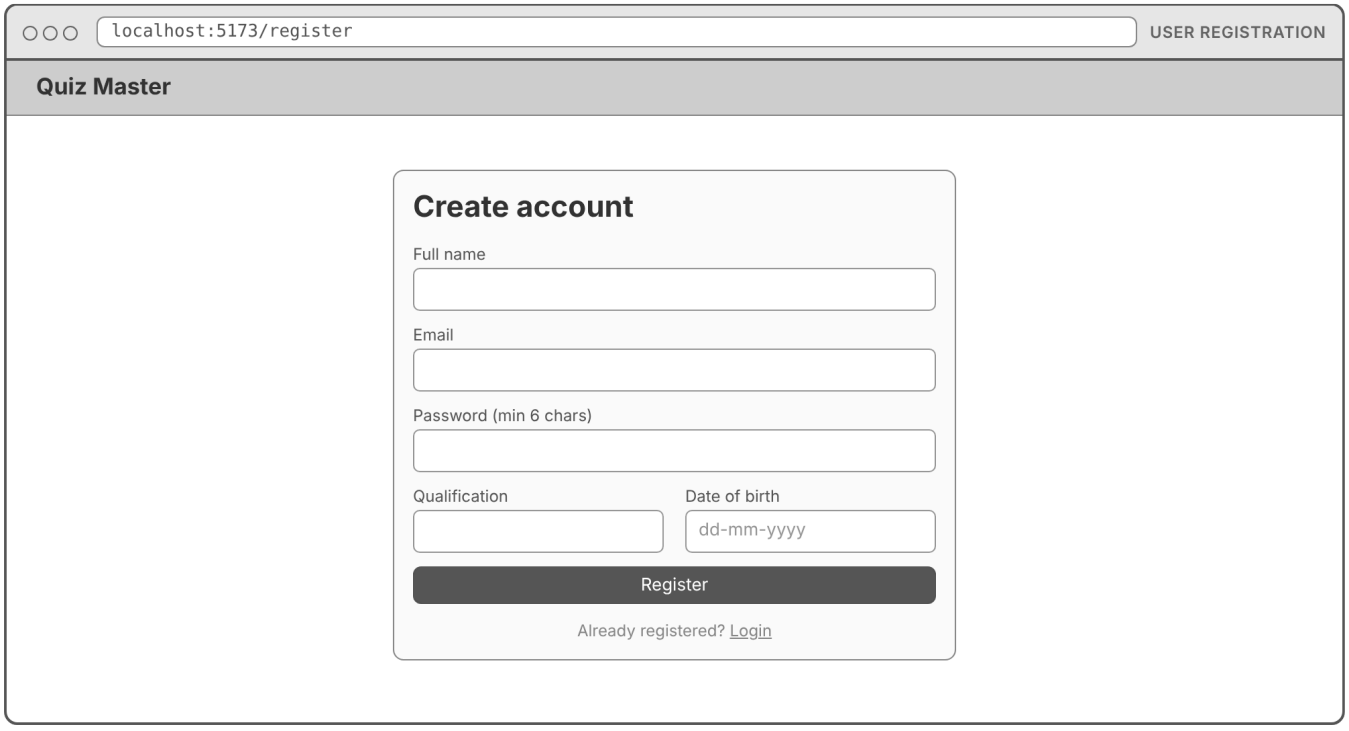


1. Login (admin and user)

One form for both roles; the app opens the right dashboard after login.

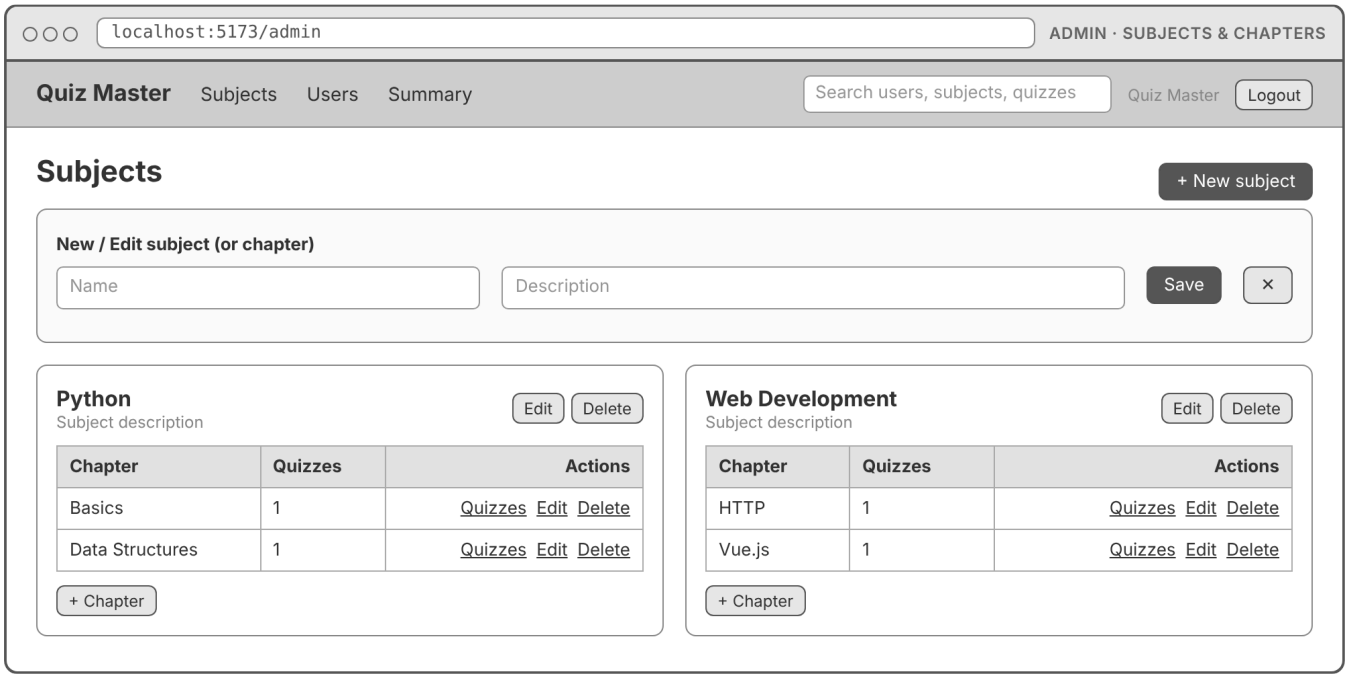


2. Register (user only)



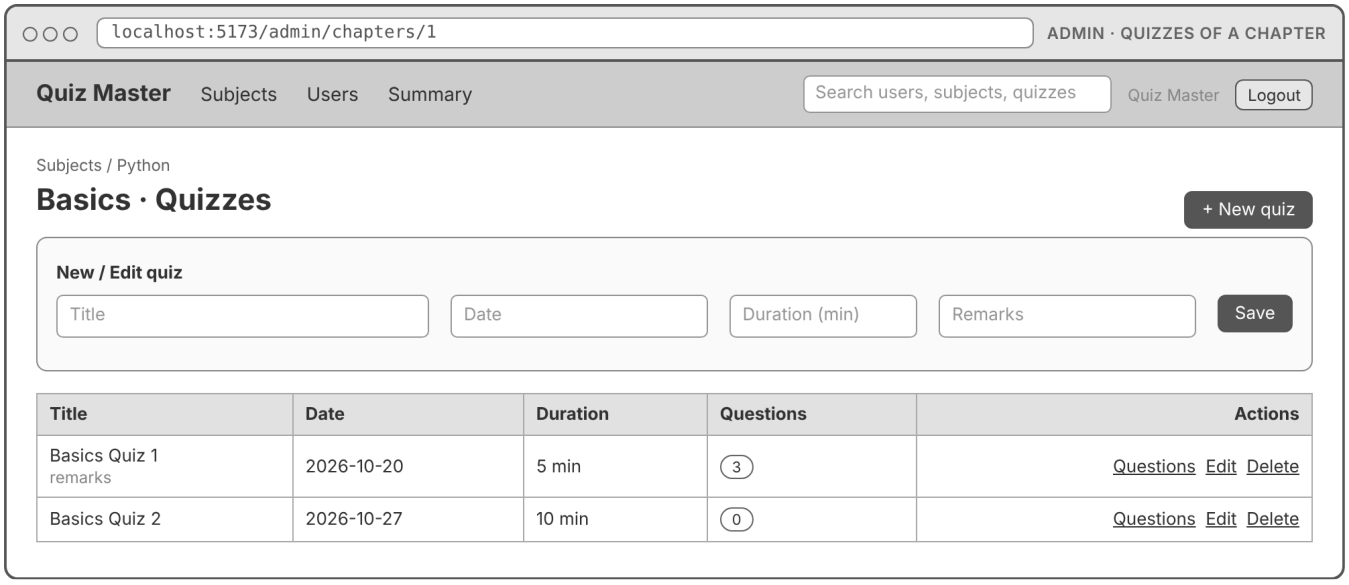
3. Admin · Subjects and chapters (admin home)

Each subject is a card with its chapters. One small form is used to add or edit a subject or a chapter.



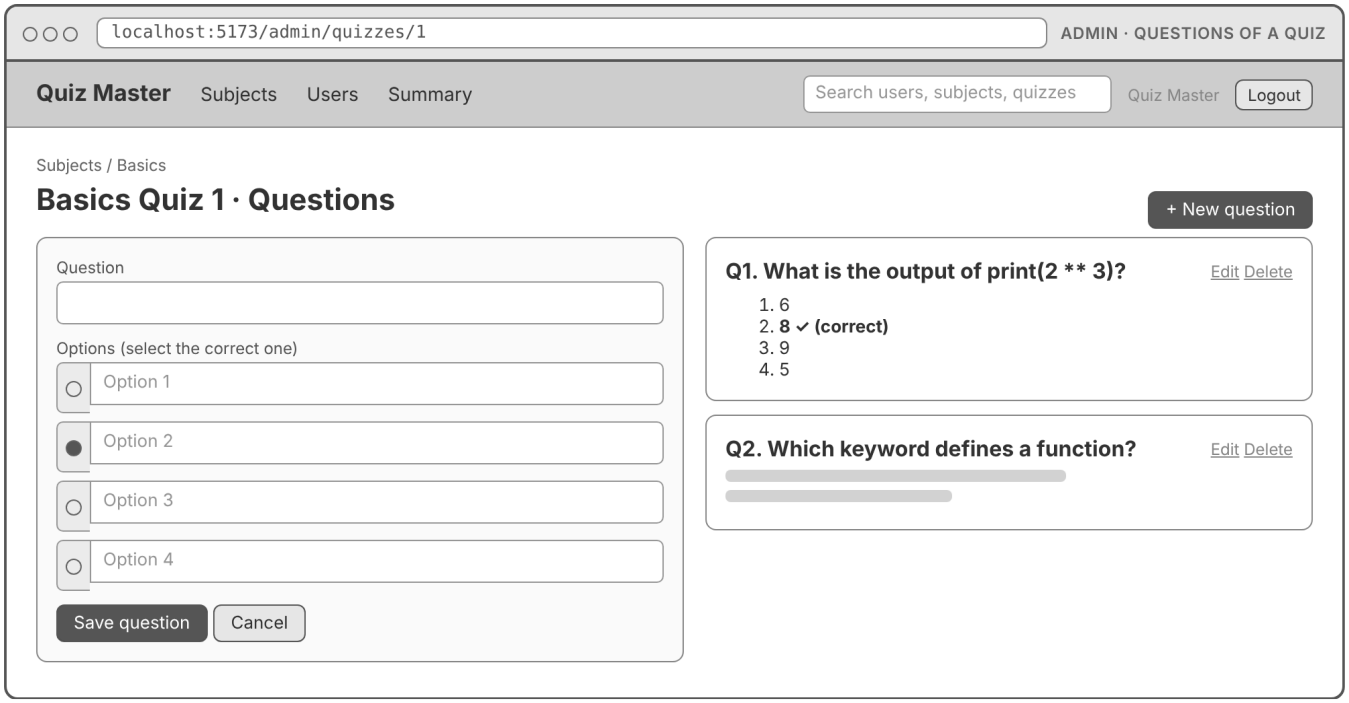
4. Admin · Quizzes of a chapter

Add or edit a quiz with title, date, duration and remarks; open its questions.



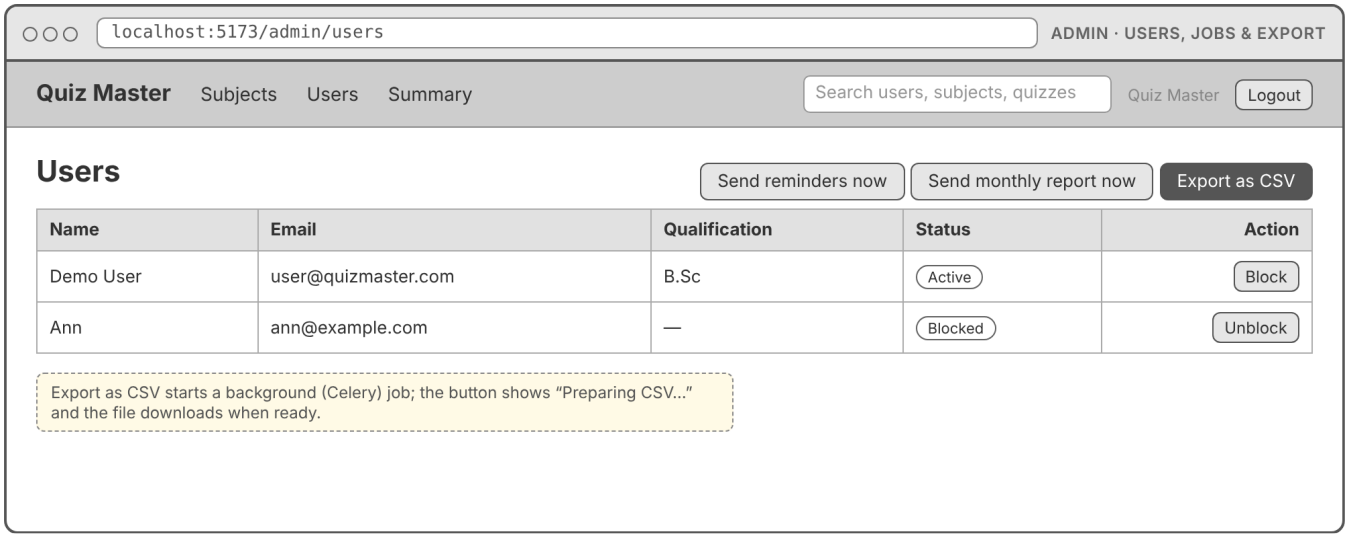
5. Admin · Questions of a quiz

Question text, up to 4 options, a radio button marks the one correct option.



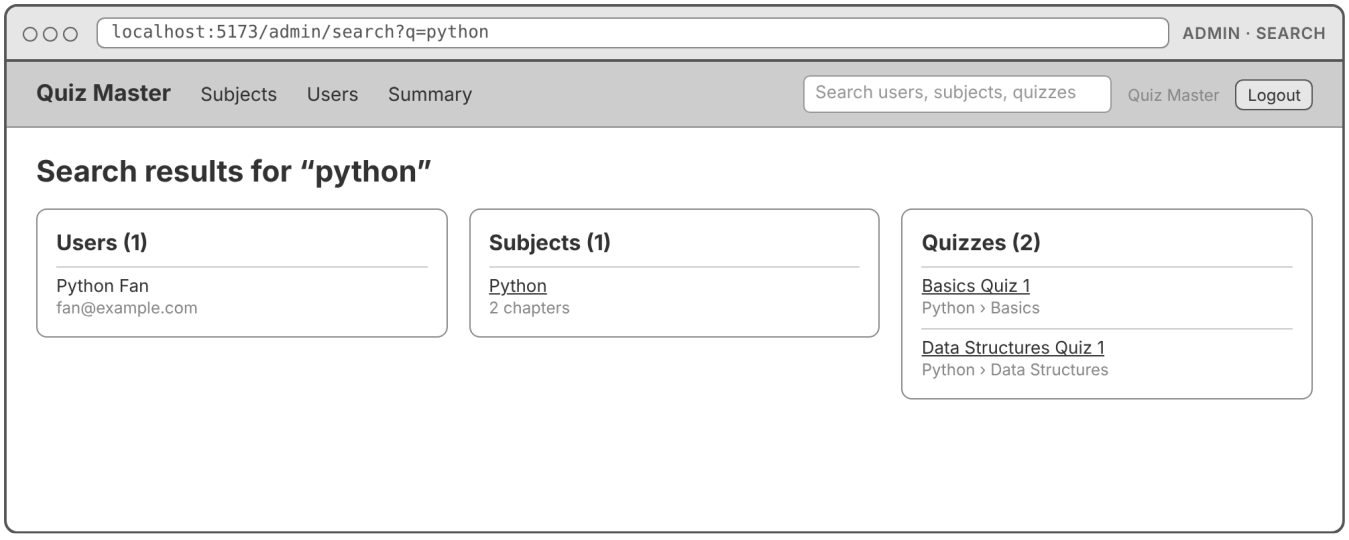
6. Admin · Users, jobs and export

Block or unblock users, run the reminder and monthly report jobs, export user statistics as CSV.

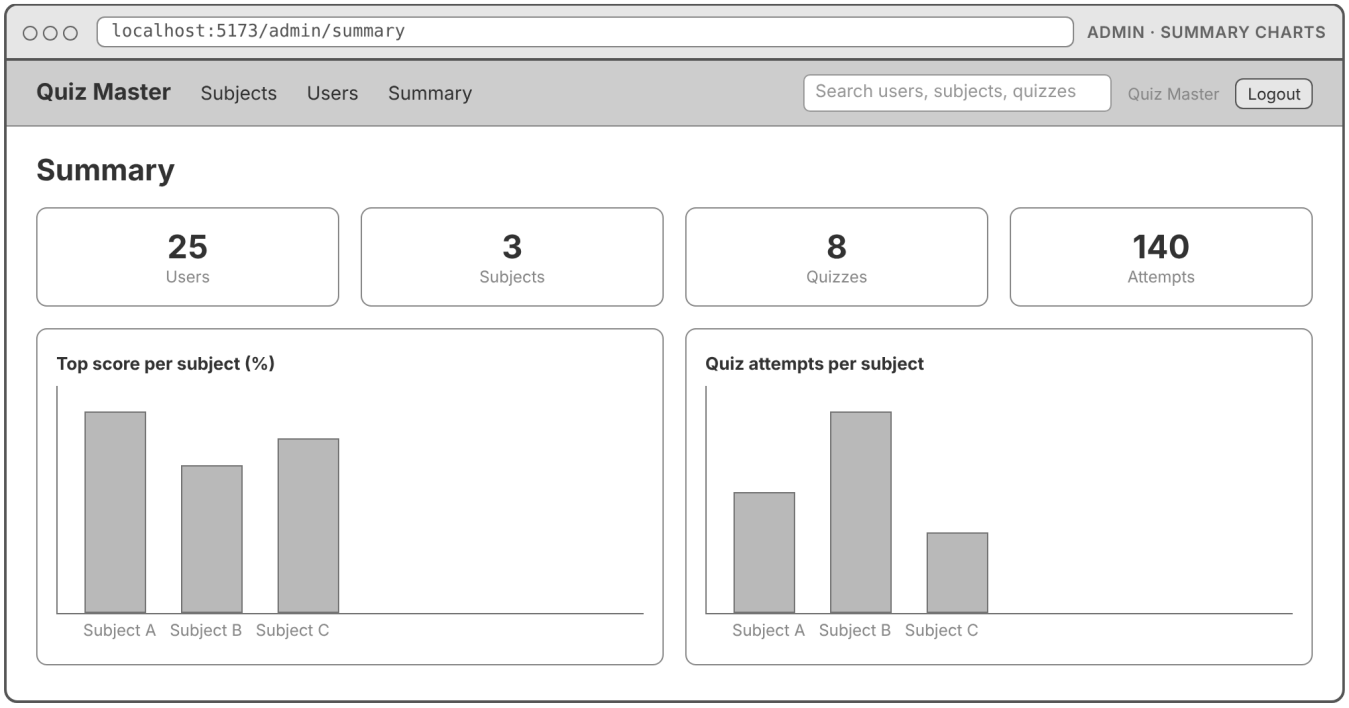


7. Admin · Search

The navbar search box shows matching users, subjects and quizzes.

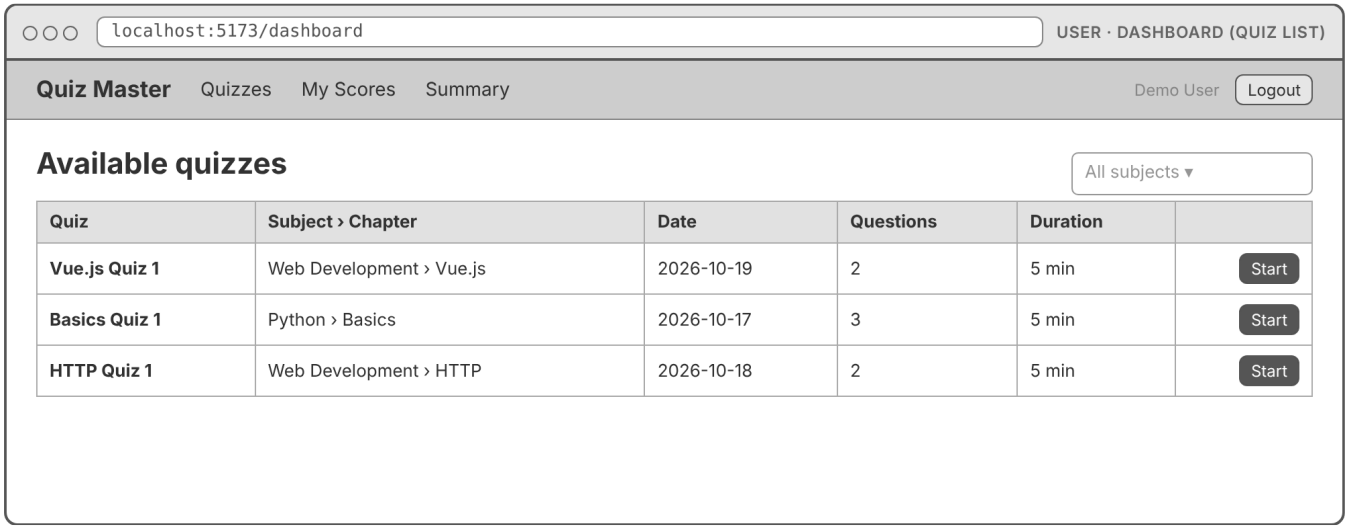


8. Admin · Summary charts



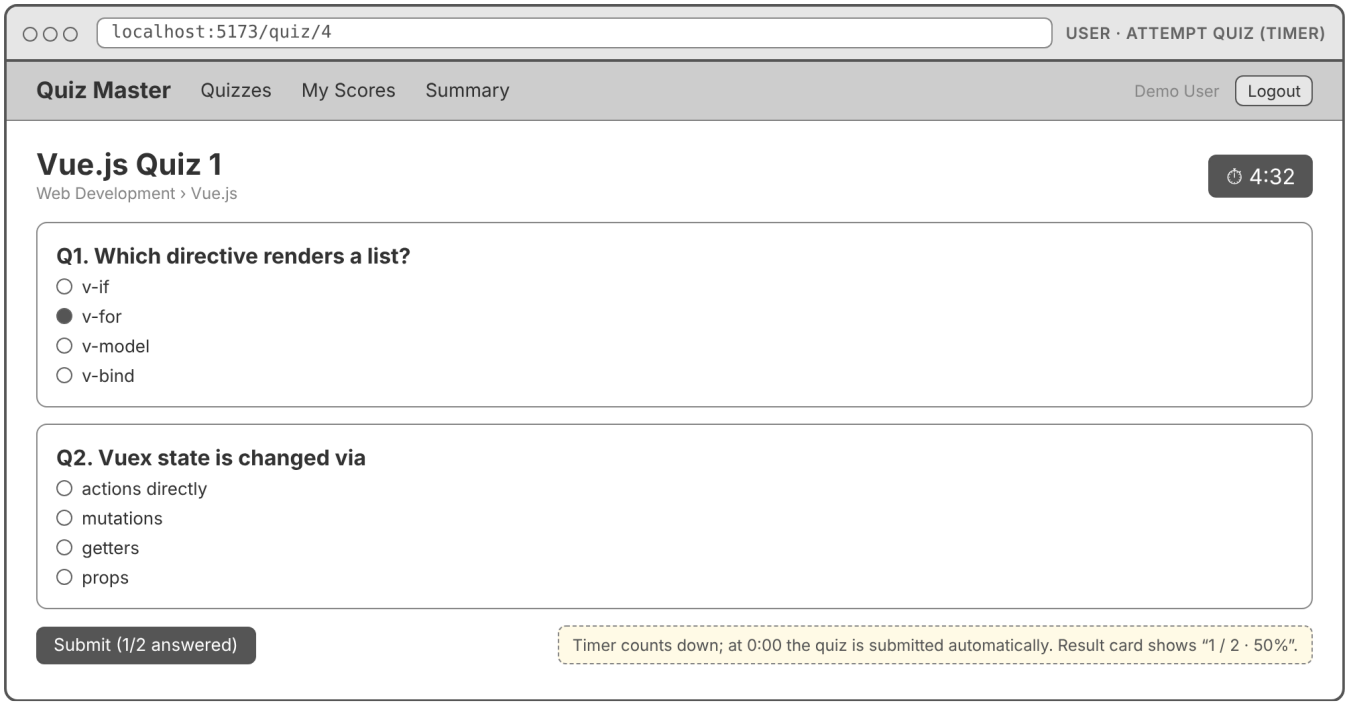
9. User · Dashboard (quiz list)

All available quizzes with a subject filter and a Start button.



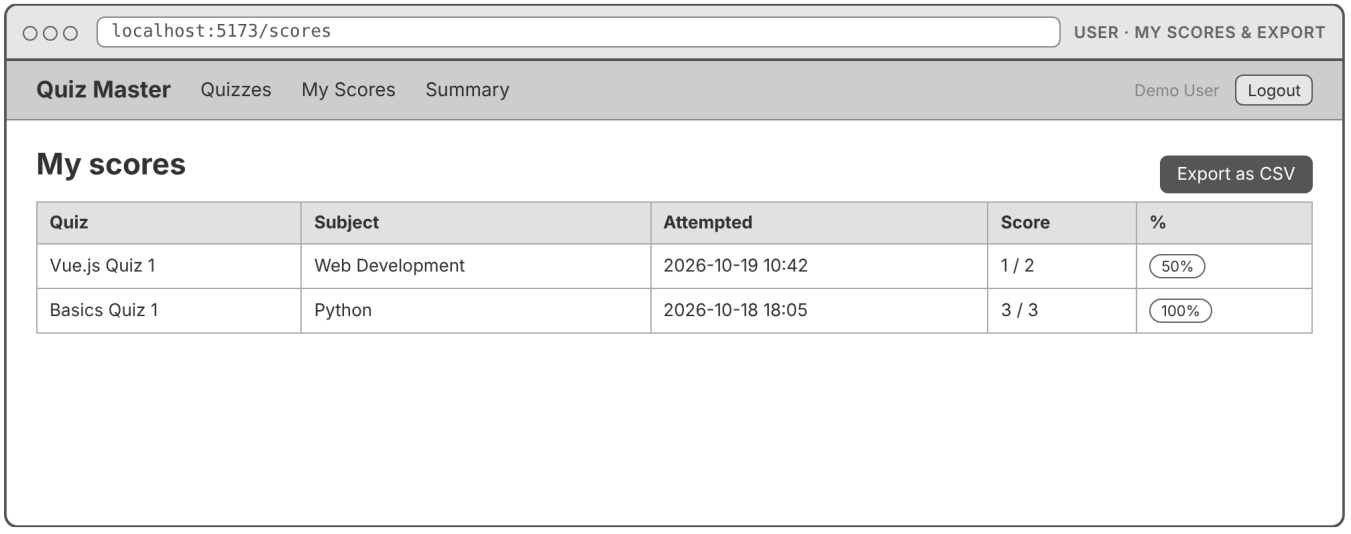
10. User · Attempt quiz

All questions on one page with a countdown timer; auto-submits when time runs out, then shows the score.



11. User · My scores

Past attempts and an Export as CSV button.



12. User · Summary charts

